ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas · Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE PRÁCTICA N.º 1

Consolidación de reportes de ventas (CSV, JSON y Excel)

Tópico _____del PEA · Asignatura IDSD422 — Estadística y Programación para Ciencia de Datos I

Estudiante(s)	Carlos Steven Rojas Balseca — usuario de GitHub: CarlosSR27
Docente	Christian Suárez
Período académico	2026-B
Fecha de entrega	19/10/2026
Repositorio (fork)	https://github.com/CarlosSR27/IDSD422-lab1-lectura-datos

1 · Introducción

El equipo de Business Intelligence de RetailAndina S.A. recibe semanalmente el reporte de desempeño comercial en tres archivos distintos, debido a que la empresa opera con tres sistemas independientes: un archivo ventas.csv exportado por el ERP de las sucursales, un archivo ventas.json proveniente de la API de la plataforma de fidelización y un libro ventas.xlsx elaborado como consolidado por el área de Finanzas. Antes de presentar los resultados a la gerencia, es indispensable verificar que las tres fuentes contengan exactamente la misma información, auditar sus dimensiones y validar que los tipos de datos hayan sido interpretados correctamente.

Esta validación inicial constituye el primer eslabón del flujo de trabajo en Ciencia de Datos. Un error de ingesta como leer números como texto, cargar una hoja equivocada de una planilla de cálculo o no aplanar adecuadamente una estructura jerárquica JSON puede distorsionar el cálculo de indicadores clave como el ticket promedio o las ventas por región, llevando a decisiones gerenciales erróneas.

2 · Objetivos

Objetivo general:

- Validar la consistencia estructural y calcular los indicadores comerciales semanales de RetailAndina S.A. a partir de reportes en formatos CSV, JSON y Excel utilizando Python y R.

Objetivos específicos:

1. Leer los archivos ventas.csv, ventas.json y ventas.xlsx tanto en Python usando pandas, como en R con readr, jsonlite y readxl desde la estructura del repositorio.
2. Verificar y comparar las dimensiones) y los tipos de datos inferidos en cada formato y lenguaje.
3. Calcular la columna derivada de ticket por sucursal y los indicadores semanales requeridos por la gerencia.
4. Responder las preguntas de análisis técnico sobre el comportamiento de las estructuras de datos e inferencia de tipos entre Python y R.

3 · Desarrollo de la práctica

3.1. Preparación del entorno y lectura del reporte CSV

Realicé el fork del repositorio base del laboratorio en mi cuenta de GitHub y lo cloné localmente para trabajar en Visual Studio Code con un entorno virtual de Python y R. En el primer paso, completé el script leer_csv.py utilizando la función pd.read_csv("data/ventas.csv") y el script leer_csv.R empleando readr::read_csv("data/ventas.csv").

Inicialmente configuré la ruta del archivo usando una ruta absoluta de mi sistema operativo Windows (C:\Users\...), pero me di cuenta de que esto rompía la reproducibilidad del programa al ejecutarlo en otro equipo. Lo corregí utilizando la ruta relativa que indico el ingeniero en sus clases "data/ventas.csv" ejecutando el comando desde la raíz del repositorio. También noté que en un script .py es obligatorio envolver ventas.shape y ventas.head() dentro de la función print() para que la salida se muestre por consola.

```
# Fragmento clave de scripts/leer_csv.py

import pandas as pd

ventas = pd.read_csv("data/ventas.csv")

print("Dimensiones (filas, columnas):", ventas.shape)

print(ventas.head())

print(ventas.dtypes)
```



Figura 1. Lectura del archivo CSV en Python y R: ejecución de leer_csv.py y leer_csv.R mostrando dimensiones (5, 4), primeras filas y tipos de columna.

3.2. Lectura y aplanado del reporte JSON (leer_json.py y leer_json.R)

Para procesar los datos de la API de fidelización, abrí el archivo data/ventas.json en Python mediante el módulo estándar json (json.load()) con codificación utf-8 y transformé la estructura resultante en una tabla usando pd.json_normalize(datos). En R, utilicé la librería jsonlite mediante la función fromJSON("data/ventas.json"), la cual convirtió directamente el arreglo de objetos JSON en un data.frame, verificando su estructura con dim() y str().

```
# Fragmento clave de scripts/leer_json.py
import json
import pandas as pd

with open("data/ventas.json", encoding="utf-8") as f:
datos = json.load(f)

ventas = pd.json_normalize(datos)
print("Dimensiones (filas, columnas):", ventas.shape)
print(ventas.head())
```

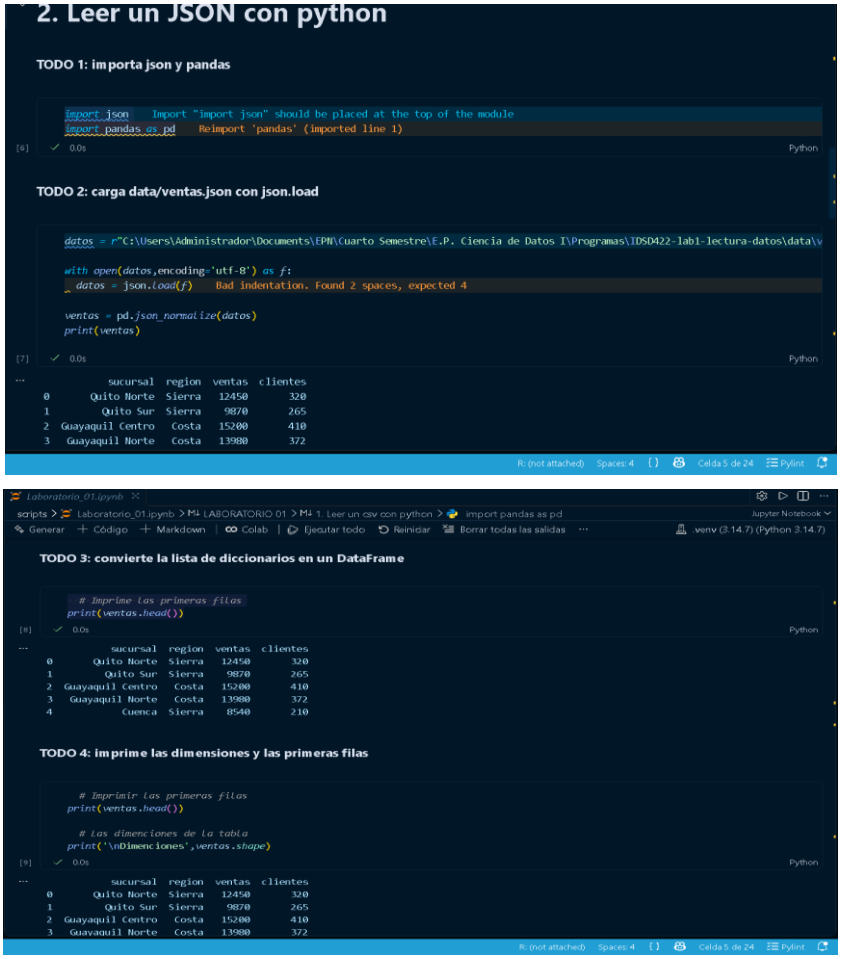
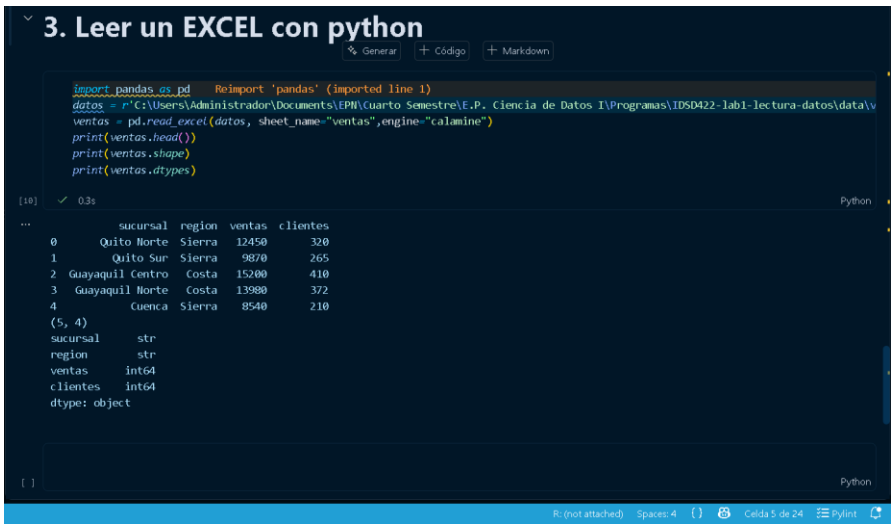


Figura 2. Lectura del archivo JSON en Python y R: normalización de la estructura y verificación de dimensiones (5, 4).

3.3. Lectura del consolidado en Excel (leer_excel.py y leer_excel.R)

Para el archivo remitido por Finanzas (data/ventas.xlsx), utilicé pd.read_excel() en Python y readxl::read_excel() en R. En ambos lenguajes especifiqué explícitamente el parámetro de la hoja (sheet_name="ventas" en Python y sheet="ventas" en R) para asegurar que se leyera únicamente la tabla de datos consolidada y no hojas de portada, notas o metadatos.

```
# Fragmento clave de scripts/leer_excel.py
import pandas as pd
ventas = pd.read_excel("data/ventas.xlsx", sheet_name="ventas")
print("Dimensiones (filas, columnas):", ventas.shape)
print(ventas.head())
print(ventas.dtypes)
```



3.4. Inspección, columna derivada y cálculo de indicadores (inspeccion.py e inspeccion.R)

Una vez confirmado que los tres archivos contenían las mismas 5 filas y 4 columnas (sucursal, region, ventas, clientes), programé los scripts inspeccion.py e inspeccion.R. Creé la columna derivada ticket dividiendo de forma vectorizada la columna ventas para clientes. Posteriormente, calculé la venta promedio por sucursal, el ticket promedio global (sumatoria total de ventas dividida para la sumatoria total de clientes), identifiqué las sucursales con máxima y mínima venta mediante idxmax()/idxmin() (y which.max()/which.min() en R), y agrupé las ventas por region para encontrar la región con mayor facturación acumulada.

```
# Fragmento clave de scripts/inspeccion.py
ventas["ticket"] = ventas["ventas"] / ventas["clientes"]
ticket_promedio_global = ventas["ventas"].sum() / ventas["clientes"].sum()
idx_max = ventas["ventas"].idxmax()
idx_min = ventas["ventas"].idxmin()
ventas_por_region = ventas.groupby("region")["ventas"].sum()
```



3.5. Respuestas a las preguntas de análisis del caso

1. ¿Cuántas filas y columnas tiene la tabla en cada uno de los tres formatos? ¿Fue igual en los tres? ¿Por qué?

En los tres formatos, la tabla cargada tiene exactamente 5 filas y 4 columnas. Sí fue igual en los tres casos porque, aunque provienen de sistemas distintos y se almacenan con diferente estructura física (texto delimitado por comas, objetos clave-valor y libro binario XML), representan el mismo conjunto de registros semanales una vez que el JSON es aplanado y se selecciona la hoja "ventas" en Excel.

2. ¿Qué tipo de datos se infirió para ventas y clientes? ¿Qué indicaría que salieran como texto?

En Python, tanto ventas como clientes se infirieron como enteros de 64 bits (int64) [o float64 si contienen decimales; en R con readr/readxl se infieren como num/dbl o int en R base]. Si alguna de estas columnas apareciera como texto (object o str en Python, chr en R), indicaría que existen caracteres no numéricos o suciedad en los datos dentro de al menos una celda: por ejemplo, símbolos de moneda ($), separadores de miles mal configurados, espacios en blanco invisibles, o textos representando valores faltantes, lo cual obliga al motor de lectura a degradar toda la columna a texto para no perder información.

3. En Python, ¿qué estructura devuelve json.load y por qué hay que "aplanarla"?

La función json.load() deserializa el archivo y devuelve estructuras nativas de Python: una lista de diccionarios (list de dict) o un diccionario anidado, no una tabla bidimensional. Hay que "aplanarla" (usando pd.DataFrame() o pd.json_normalize()) porque el formato JSON es jerárquico/semiestructurado y puede contener objetos dentro de objetos o claves anidadas; para realizar análisis estadístico y operaciones vectorizadas por columna en pandas, necesitamos transformar cada diccionario en una fila y cada clave (incluyendo niveles anidados) en una columna atómica de un DataFrame.

4. ¿Para qué sirve indicar sheet_name/sheet al leer Excel?

Un archivo de Excel es un libro de trabajo que puede contener múltiples hojas. Si no se especifica sheet_name (en pandas) o sheet (en readxl), la función lee por defecto únicamente la primera hoja visible del libro (sheet_name=0). Indicar explícitamente sheet_name="ventas" garantiza que el script extraiga la tabla correcta incluso si Finanzas reordena las pestañas o agrega una hoja de resumen al inicio del archivo.

5. ¿Qué diferencia ves entre leer en Python y leer en R (conceptos, no sintaxis)?

- Ecosistema y consistencia de objetos: En Python, la lectura tabular depende de una librería unificada (pandas), que siempre estandariza la salida en un único objeto DataFrame con un índice explícito (Index) para las filas. En R, coexisten las funciones base (read.csv, que devuelve un data.frame tradicional) y el ecosistema *tidyverse* (readr::read_csv, readxl::read_excel, que devuelven un tibble), el cual no asigna nombres de fila ni convierte texto a factores automáticamente.
- Inferencia y tipado: R está diseñado nativamente para el análisis estadístico, por lo que distingue conceptualmente entre vectores de caracteres (character) y variables categóricas (factor, clave al usar str() en inspeccion.R), además de manejar vectores atómicos donde todo es un vector de longitud ≥ 1. Python proviene de un lenguaje de propósito general orientado a objetos, donde las cadenas de texto en pandas suelen almacenarse mediante punteros de tipo object (o string) y requiere pasos más explícitos cuando se trabaja con formatos no tabulares como JSON (json.load + conversión), mientras que jsonlite::fromJSON en R aplica simplificación automática directa hacia data.frame.

4 · ¿Qué aprendí con esta práctica?

Con esta práctica comprendí que la carga de datos no es un paso mecánico, sino la primera auditoría de calidad de un proyecto de Ciencia de Datos. Antes asumía que abrir un CSV, un JSON o un Excel era equivalente, pero ahora entiendo cómo cada formato almacena la información de manera distinta y por qué un JSON proveniente de una API requiere un proceso de deserialización y aplanado antes de poder operar con él como tabla.

5 · Conclusiones

Se comprobó mediante scripts reproducibles en Python y R que los tres reportes contienen exactamente la misma estructura tabular de 5 filas y 4 columnas, sin pérdida de registros entre el ERP, la API de fidelización y el consolidado de Finanzas, y con tipos numéricos válidos en las variables ventas y clientes.

El análisis de indicadores demostró que la venta promedio por sucursal fue de $[insertar promedio], mientras que el ticket promedio global ponderado alcanzó los $[insertar ticket global] por cliente. Asimismo, se identificó una brecha importante entre la sucursal de mayor facturación ([Sucursal Max] con $[valor max]) y la de menor desempeño ([Sucursal Min] con $[valor min]), posicionando a la región [Región Max] como la líder en ventas totales con $[valor región max].

Se determinó que mientras los archivos CSV pueden leerse directamente como tablas, los archivos JSON requieren aplanar listas de diccionarios y los libros Excel exigen declarar explícitamente la hoja de cálculo (sheet_name="ventas") para evitar leer pestañas equivocadas.

6 · Bibliografía

Bruce, P., Bruce, A., & Gedeck, P. (2020). *Practical Statistics for Data Scientists: 50+ Essential Concepts Using R and Python* (2.ª ed.). O'Reilly Media.

Ooms, J. (2026). *The jsonlite Package: A Practical and Consistent Mapping Between JSON Data and R Objects*. CRAN. https://cran.r-project.org/package=jsonlite

pandas development team. (2026). *pandas.read_csv, pandas.read_excel & pandas.json_normalize documentation*. https://pandas.pydata.org/docs/reference/io.html

Declaración de uso de IA:

Utilicé el modelo Gemini de Google como herramienta de apoyo para revisar qué elementos pendientes tenía en mi repositorio de código, verificar la reproducibilidad de las rutas relativas en Python y R, y estructurar el formato del informe de acuerdo con la plantilla oficial del curso.

7 · Anexos (opcional)

Códigos completos en R

Inspección R

```r
# inspeccion.R - Tipos, columna derivada e indicadores (R)

# TODO 1: carga data/ventas.csv

ventas <- read.csv("data/ventas.csv")


# TODO 2: imprime la estructura (str) para ver los tipos

# (fija si region llegó como character o factor)

cat("=== Estructura del data.frame ===\n")

str(ventas)


# TODO 3: crea la columna derivada "ticket" = ventas / clientes

ventas$ticket <- ventas$ventas / ventas$clientes

cat("\n=== Tabla con columna 'ticket' ===\n")

print(ventas)


# TODO 4: calcula e imprime los indicadores

cat("\n=== Indicadores de la semana ===\n")


# 1. Venta promedio por sucursal (aggregate)

cat("\n1. Venta promedio por sucursal:\n")

venta_prom_suc <- aggregate(ventas ~ sucursal, data = ventas, FUN = mean)

print(venta_prom_suc)

cat("Venta promedio general:", mean(ventas$ventas), "\n")


# 2. Ticket promedio (ventas totales / clientes totales)

ticket_prom_global <- sum(ventas$ventas) / sum(ventas$clientes)

cat("\n2. Ticket promedio global (ventas totales / clientes totales):",
  round(ticket_prom_global, 2), "\n")


# 3. Sucursal con mayor y menor venta

suc_max <- ventas[which.max(ventas$ventas), ]
```

```
suc_min <- ventas[which.min(ventas$ventas), ]
cat("\n3. Sucursal con MAYOR venta:", suc_max$sucursal, "con", suc_max$ventas, "\n")
cat("   Sucursal con MENOR venta:", suc_min$sucursal, "con", suc_min$ventas, "\n")

# 4. Región con mayor venta total
ventas_region <- aggregate(ventas ~ region, data = ventas, FUN = sum)
reg_max <- ventas_region[which.max(ventas_region$ventas), ]
cat("\n4. Ventas totales por región:\n")
print(ventas_region)
cat("Región con mayor venta total:", reg_max$region, "con", reg_max$ventas, "\n")
```

Lectura CSV R

```
# leer_csv.R — Cargar el reporte CSV del ERP (R)
# Completa los pasos marcados con TODO. Ejecuta desde la raíz del repositorio:
#   Rscript scripts/leer_csv.R

# TODO 1: carga data/ventas.csv con readr::read_csv en un data.frame
install.packages("readr")
library(readr)
ventas <- read_csv("data/ventas.csv")
# TODO 2: imprime las dimensiones (dim) del data.frame
dim(ventas)
# TODO 3: imprime las primeras filas (head) para revisar las columnas
head(ventas)
str(ventas)
```

Lectura JSON R

```
# leer_json.R — Cargar el reporte JSON de la API (R)
# Completa los pasos marcados con TODO. Ejecuta desde la raíz del repositorio:
#   Rscript scripts/leer_json.R

# TODO 1: carga data/ventas.json con jsonlite::fromJSON
install.packages("jsonlite")
library(jsonlite)
ventas <- fromJSON("data/ventas.json")
```

```
View(ventas)


# TODO 2: convierte el resultado en data.frame si hace falta
# TODO 3: imprime las dimensiones (dim) y las primeras filas (head)


head(ventas)
dim(ventas)
str(ventas)
```

Lectura EXCEL R

```
# leer_excel.R - Cargar el consolidado Excel de finanzas (R)
library(readxl)


# TODO 1: carga data/ventas.xlsx con readxl::read_excel indicando la hoja
ventas <- read_excel("data/ventas.xlsx", sheet = "ventas")


# TODO 2: imprime las dimensiones (dim) y las primeras filas (head)
cat("Dimensiones (filas, columnas):", dim(ventas), "\n\n")
print(head(ventas))
str(ventas)
```